GyroPrompt User Manual

Introduction

GyroPrompt is a command-line scripting language that enables the automation of higher level tasks. GyroPrompt's primary focus is easing the transfer of data over networks via simplified steps, while also providing users a robust environment to create in. GyroPrompt is very simplified, almost as simplified as BASIC. The main difference you may notice is GyroPrompt is a case sensitive language and requires a lot of capitalization compared to other scripting languages.

Table of Contents

Working with Variables…….2
Variables: String Manipulation...….. 5
Variables: Math Operations…….. 6
Working with Lists……7
Working with the File System…….. 8
Working with Timed Functions…… 10
Working with Graphical Prompts…..11
Working with the Serial Port…….12
Basic Network Commands……13
Working with Blockchain……. 14

This user manual has been shipped with the Beta 1.0.1.0 release of GyroPrompt Scripting Language.
This is not an official build. Please report bugs, issues, and suggestions to: gyrosoftlabs@outlook.com

Working with Variables

There are two primary variable types, strings (STR) and integers (INT). The syntax for declaring a new variable looks like this: **NEW STR/INT Name Value**

Your declarations, **NEW** and the variable type **STR/INT** must be capitalized, the name and values do not. In the picture below you see the variable name encapsulated in brackets – be advised that you don't always need to encapsulate when referencing or using a variable. Sometimes you must put an underscore before the variable name, other times you do not need any type of indicator that you are using an existing variable. That will all be explained shortly.

The **PRINT** command has 7 reserved one letter tags in it. See the table and image below.

```
GyroPrompt Scripting Language.

GyroPrompt Script > NEW STR StringVariable This is the message.
GyroPrompt Script > NEW INT OneThousand 1000
GyroPrompt Script > PRINT {R}{OneThousand}{Y}{StringVariable}{L}{B}Time: {T}{W}Date: {D}
1000This is the message.
Time: 11:59 AMDate: 09/10/2020
GyroPrompt Script > PRINT {R}Red{L}{W}White{L}{B}Blue{L}{Y}Yello{L}{T} {D}
Red
White
Blue
Yello
12:00 PM 09/10/2020
GyroPrompt Script >
```

You will not be able to declare a new variable with any of the following reserved values:

Letter	Value
R	Color Red
W	Color White
B	Color Blue
Y	Color Yellow
L	New Line
T	Time
D	Date

Name	Value
WINDOWHEIGHT	The console's height.
WINDOWWIDTH	The console's width.
CURSORX	The X coordinate of the cursor.
CURSORY	The Y coordinate of the cursor.
BUFFERDELAY	Pause between lines in a script in MS.

In order to compare a variable's value and execute a command with the result via an IF statement, your syntax should look something like this: **IF VARIABLE = VALUE THEN COMMAND**
You can use the equal sign = to see if the values match, or an exclamation-equal != symbol to see if the values do not match. When using an IF statement you are not required to encapsulate the first variable or use an underscore indicator, but you should underscore the second variable. Mathematical operates such as greater than > less than < greater than or equal to >= and less than or equal to <= work if both variables are integers. For now we'll focus on simple statements.

```
GyroPrompt Script > NEW INT ExampleNumber 50
GyroPrompt Script > IF ExampleNumber = 50 THEN PRINT Matches!
Matches!
GyroPrompt Script > IF ExampleNumber != 50 THEN PRINT Matches!
GyroPrompt Script > IF ExampleNumber != 49 THEN PRINT Matches!
Matches!
GyroPrompt Script > _
```

Prompting a user for input, to have the input placed into a variable, is extremely simple with the **INQUIRY** command. The inquiry command will automatically filter the input based on the inquiry code, which must range from 0 to 2. The command's syntax looks like this:
INQUIRY 0/1/2 VARIABLE PROMPT

```
GyroPrompt Scripting Language.

GyroPrompt Script > NEW INT FirstBoolean 0
GyroPrompt Script > NEW STR UserMessage Nothing.
GyroPrompt Script > INQUIRY 0 _FirstBoolean Are gyros delicious?
Are gyros delicious? (Y/N) i
Are gyros delicious? (Y/N) o
Are gyros delicious? (Y/N) u
Are gyros delicious? (Y/N) Y
GyroPrompt Script > INQUIRY 1 _UserMessage Please type response
Please type response: What's up?

GyroPrompt Script > PRINT {FirstBoolean} then {UserMessage}
1 then What's up?
GyroPrompt Script > INQUIRY 2 Value Year you were born
Year you were born: 1990

GyroPrompt Script > IF Value = 1990 THEN PRINT {R}Thirty years!
Thirty years!
GyroPrompt Script >
```

The inquiry code determines the kind of input being requested from the user. 0 means a Y/N question. 1 means a string message, and 2 means a number. The input will automatically be filtered.

You can use an existing variable, in which case you will use an underscore to indicate the variable to already exist. If there is no underscore, the inquiry will create a new variable automatically and assign

the user's input to its value. The variable will be a string, even for the Y/N boolean inquiry. You can use inquiry code 0 and assign it to a string variable *and* an integer variable. If the variable is an integer it will adopt 0 for No and 1 for Yes. If the variable is a string it will adopt 0 or 1 as the new value as well.

Code	Inquiry
0	Boolean Y/N Question
1	User Types a String
2	User Inputs a Number

The **SET** command is a command used to reassign a variable's value. You do not need to use an underscore indicator or encapsulate when using SET, and the syntax is incredibly simple, but you should encapsulate any variables in the VALUE:

SET VARIABLE = VALUE

Command	Function
TERMINATE	Ends the program.
CLEAR	Clears the console of all text.
PROMPT	(ON/OFF/TOGGLE) Change console prompt.
TITLE	Changes console title.
RUN	Run, followed by a file name, executes the file as script.
GOTO	While running a script, will move to specified line number.
EXEC	While running a script, will move to specified function.
ADD1	Increments and integer by 1 single tick.
SUB1	Negates an integer by 1 single tick.
SETCURSOR	Sets the cursor position to the specified X Y coordinate or ORIGIN.
TOGGLE	Toggle variable being treated as booleans (1 into 0, 0 into 1)
SLEEP	Pauses the script for specified time in milliseconds.

Functions are a one word declaration beginning with a colon. In the example script to the right, the prompt prints "Hello" 5 times in a loop then terminates. The EXEC commands directs to the location of the function START.

```
NEW INT Count 0
:START
PRINT Hello
ADD1 Count
IF Count > 4 THEN TERMINATE
EXEC START
```

Variables: String Manipulation

String variables have 4 command sets purposed for the manipulation and management of the string's value. With these 4 commands you can append, remove, and replace values within a specified string. You can also check if a string contains a specified value and return either a boolean response, or the number of times the specified value exists.

STRING APPEND VARIABLE VALUE
STRING _APPEND VARIABLE VALUE

Where **VARIABLE** is your string name and **VALUE** is the information being added to your string, the underscore modifier in front of **APPEND** tells the prompt whether or not to process all bracketed values. So if there is a time tag {T} in **VALUE**, with **_APPEND** it will be converted to the tag value (the current time) but without the underscore modifier, **APPEND** will keep it as {T} without the tag conversion.

STRING REMOVE VARIABLE VALUE

Simple. Removes the **VALUE** from **VARIABLE** if found.

STRING REPLACE VARIABLE FIND_THIS||REPLACE_WITH_THIS

Searches **VARIABLE** and replaces **FIND_THIS** with **REPLACE_WITH_THIS**. It is important to note that you must use two pipes || to separate the searched value and the replacing value.

STRING CONTAINS VARIABLE BOOL_OUTPUT VALUE
STRING CONTAINS VARIABLE COUNT_OUTPUT VALUE

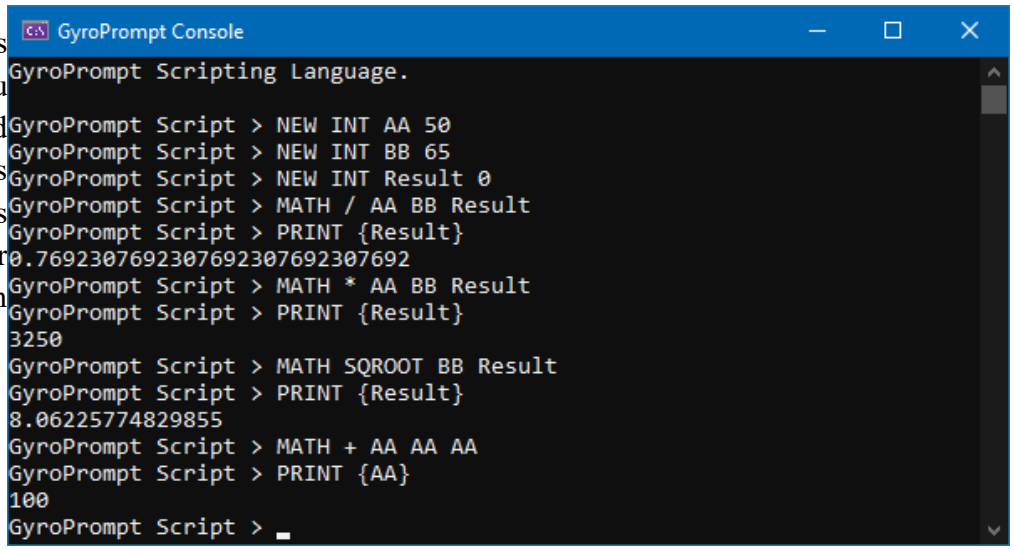
Searches **VARIABLE** for **VALUE**, then returns a value to the **OUTPUT** variable. If the output variable is preceded by **BOOL_** then the value returns 0 for no instances found and 1 for instances found. If the output variable is preceded by **COUNT_** then the output variable has its value set to the number of instances **VALUE** was found in **VARIABLE**.

Variables: Math Operations

All mathematical operations begin with **MATH**, followed by the function identifier, the variables being used, and always finalized with the output variable (where the result of the mathematical operation is put). Below is a table of mathematical operations.

Command Syntax					Description
MATH	+	VARIABLE	VARIABLE	OUTPUT VAR	Adds both variables – result into output var.
MATH	-	VARIABLE	VARIABLE	OUTPUT VAR	Adds both variables – result into output var.
MATH	/	VARIABLE	VARIABLE	OUTPUT VAR	Adds both variables – result into output var.
MATH	*	VARIABLE	VARIABLE	OUTPUT VAR	Adds both variables – result into output var.
MATH	SQROOT	VARIABLE	OUTPUT VAR		Square root of variable into output var.
MATH	COMP	VARIABLE	VARIABLE	OUTPUT VAR	Compares both variables, larger is output var.

Mathematical operations always require an output variable. You can use one of the referenced variables as the output variable as well. Your referenced variables must be integers, however your output variable can be both an integer and a string.



RANDOMIZEINT allows you to set an integer to a random number within a specified range.
For example, the below code would take the variable **INTEGER** and set it to a number between 1 and 99. You can also use bracketed variables as in place of numbers when defining the random number generator's range.

RANDOMIZEINT INTEGER 1 99

Working with Lists

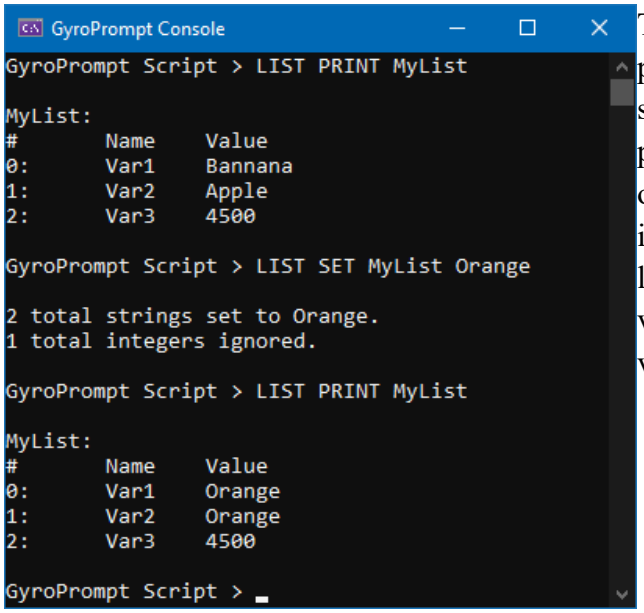
Lists are fluid variable containers. Lists can hold strings and integers. Below is a table containing all list commands.

Command Syntax				Description
LIST	NEW	NAME		Creates a new list.
LIST	ADD	NAME	VARIABLE	Appends specified variable to list.
LIST	REMOVE	NAME	VARIABLE	Removes specified variable from list.
LIST	PRINT	NAME		Prints list in human-readable table.
LIST	SET	NAME	VALUE	Sets all variables in list to value.

When referencing a list within a command such as PRINT you must differentiate with a pound sign while encapsulating. This is to prevent errors resulting from duplicate names existing between lists and variables. It is also possible to reference a specific variable within a list.

PRINT {#List} produces the same output as **LIST PRINT List**

PRINT {List_Variable} prints the value of the specified variable within the list. The underscore is used to separate the list from the target variable.



The **LIST SET** command sets all variables to the provided value. If the provided value is all digits, all string *and* integers will adopt the new value. If the provided value *is not* all integers and contains letters or symbols, *only* strings will adopt the new value and integers will be ignored. In the example image to the left, if you replaced Orange with a number, all three variables in the list would have adopted the new value.

Working with the File System

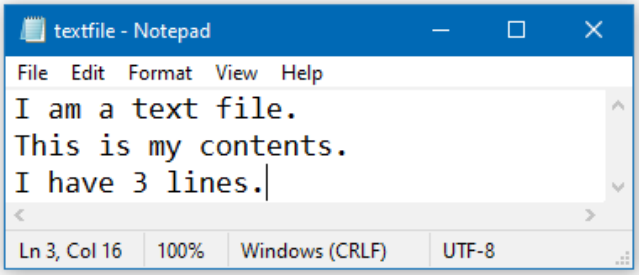
Any command for reading, checking, creating, moving, or deleting files and directories will lead with **FILESYSTEM**. FILESYSTEM tells the prompt we will be interacting with the file system. See the table below a list of the basic FILESYSTEM commands and their syntax.

Command Syntax				Description
READ	DIGIT/ALL	FILE PATH	VARIABLE	Read content of file, write to specified variable. Second parameter is either digit (no. of lines) or is **ALL** telling command to read all lines of text.
WRITE	FILE PATH	DATA		Write to and append the end of the file with the data, or if no file exists, create the file and write the data to file.
DEL	FILE PATH			Deletes specified file.
MKDIR	FILE PATH			Creates specified directory.
RMDIR	FILE PATH			Deletes specified directory.
MVDIR	FILE PATH	DESTINATION		Moves specified directory to the destination location.
GETSIZE	VARIABLE	FILEPATH		Sets variable to size of file in bytes.

To the right is an example script that creates a loop that executes 15 times. Each time the loop function runs, the time prints to the console and an entry is made into a log file. As you can see, if you are writing to a file, variables and tags should be encapsulated in brackets. The {L} tag, which prints a new line in the console, is also compatible in writing lines to a file and acts as a new line indicator. Date, time, and new line tags work when writing a file.

```
NEW INT Counter 0
:START
PRINT Begin counting.
INQUIRY 1 Username Please enter a name
FILESYSTEM WRITE Logfile.txt Entry# Time      Description{L}
:PRINTLOG
PRINT The time is {T}.
FILESYSTEM WRITE Logfile.txt {Counter}:{T}:{Username}'s session.{L}
ADD1 Counter
IF Counter < 16 THEN EXEC PRINTLOG
:END
TERMINATE
```

Next we'll look at write's counterpart, read. There are two ways in which the read command can be implemented, and it's pointing to a specific line number, or telling the prompt to read all lines. Take a look at the following examples.



FILESYSTEM READ 2 C:\folder\textfile.txt Content

The above command will create and set the variable Content to the second line of the file.

FILESYSTEM READ ALL C:\folder\textfile.txt Content

The above command will create and set the variable Content to contain the entire text of the file (all lines).

NEW INT COUNT 1
FILESYSTEM READ _COUNT C:\folder\textfile.txt Content

The above command will create and set the variable Content to contain line 1 of the file – or whatever the value is of the integer converted into a line number.

Anytime you use the **RMDIR** command or **DEL** command it is important to be aware there are no protections in place or prompts to confirm this action. If you delete a directory, you will automatically be deleting all sub directories and files contained within the specified directory.

NEW INT fileSize 0
FILESYSTEM GETSIZE fileSize C:\file.txt
PRINT File is {fileSize} bytes!

The above command would grab the size of the file and set the variable fileSize to the numerical value.

Working with Timed Functions

Timed functions can be incredibly useful tools. Think of them like background processes that run parallel to your script. There can be some great benefits to using timed functions but they do have some limitations and should be implemented carefully or they can disrupt the smooth flow of the program being executed. Not everything requires a timed function, but they can be extremely useful.

Command Syntax				Description
TASK	NEW	NAME	INTEGER	Creates a new timed function where name becomes its name and the integer becomes its turnaround in milliseconds.
TASK	ADD	NAME	COMMAND	Adds the command to the specified task's series of commands.
TASK	START	NAME		Initiates the timer on specified function.
TASK	STOP	NAME		Kills the timer on the specified function.
TASK	ADJ	NAME	INTEGER	Changes the timed function's timer turnaround after timed function has already been created.

Once you create a new timed function, you must manually enter each command it will execute in chronological order, then start it. You can enter several commands or just one command.

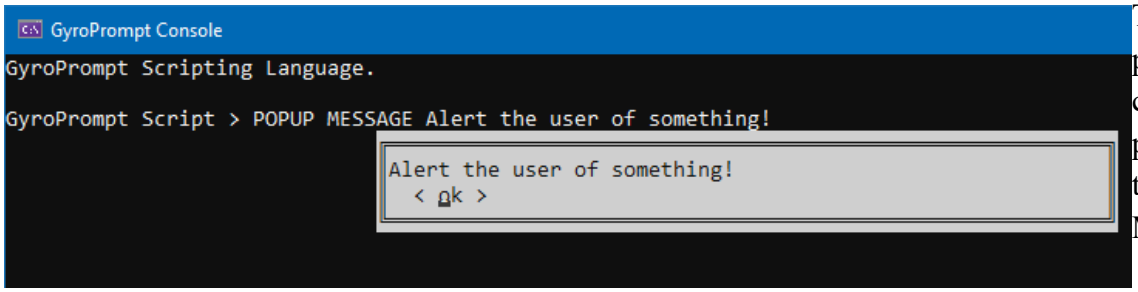
Here's an example of a script that utilizes a timed function that writes the time to Log.txt every second. It runs in the background while asking the user if they are ready to stop the logger.

```
TASK NEW Logger 1000
TASK ADD Logger FILESYSTEM WRITE Log.txt Logging time: {T}{L}
NEW INT Ticker 0
NEW INT YesNo 0
PRINT {L}Logger will make entry every second.{L}
:START
INQUIRY 0 _YesNo Start logger?
IF YesNo = 1 THEN TASK START Logger
IF YesNo = 0 THEN EXEC START
:STOP
INQUIRY 0 _YesNo Stop logger now?
IF YesNo = 1 THEN TERMINATE
IF YesNo = 0 THEN EXEC STOP
```

Again, it should be noted that timed functions should be implemented carefully. If this timed function had a PRINT command, it could interfere with the INQUIRY loop being utilized. Timed functions are more useful for background processes.

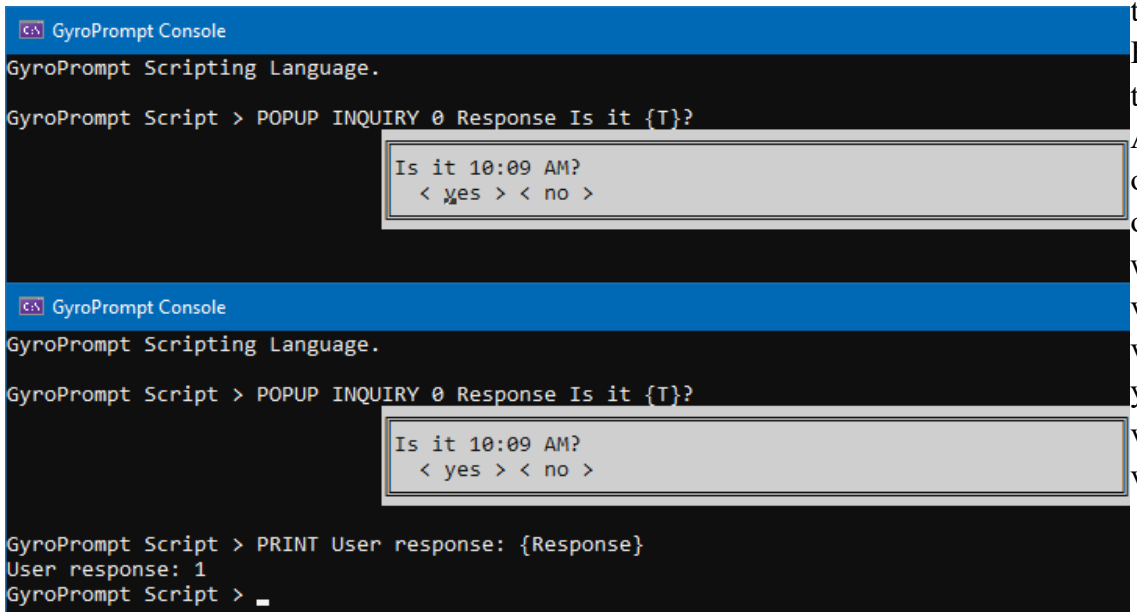
Working with Graphical Prompts

Graphical prompts create a more engaging and pleasant experience for the user. The **POPUP** command precedes any graphical prompt. You can POPUP an INQUIRY or a MESSAGE. Using POPUP for an INQUIRY utilizes the same exact syntax as INQUIRY without being proceeded by POPUP, you can refer to page 3 to see about the INQUIRY command. If you POPUP MESSAGE, it works just like PRINT. Here are some examples below.



The user must press a key to continue once presented with the POPUP MESSAGE.

POPUP INQUIRY as stated works just like INQUIRY. The input will be filtered and only allow for yes or no to be selected. The user can use the left and right arrow keys to move between the two options, then press Enter to make their selection. As with all other boolean operations, no will set the variable's value to 0 and yes will set the variable's value to 1.



As of Beta 1.0.1.0 POPUP only supports inquiry code 0, boolean yes or no questions, and does not support inquiry code 1 or 2.

Working with the Serial Port

Serial ports are useful for interacting with external hardware and devices, Arduino boards are a good example. To view all immediately open and available ports you can type **SERIALPORT ALLOPEN**. If there are available ports open, you will be able to create a new serial port channel to read and write to. If there are no open ports, you will be unable to communicate or establish a new serial port channel and will receive an error. The following syntax is used to open a new serial port channel:

SERIALPORT NEWPORT PORTNAME BAUDRATE DEFAULT_BUFFER

Where port name is the open available port, your baud rate, and the default buffer is where the received data is immediately routed to. Serial port channels also have a data received event where all the assigned commands will be executed chronologically, similarly to a timed function trigger. GyroPrompt sets aside **SERIALBUFFER** as an environmental variable that will be used if the buffer parameter is set to *default*.

SERIALPORT NEWPORT COM5 9600 default
SERIALPORT ADD COM5 COLOR 0 12
SERIALPORT ADD COM5 PRINT Received from COM5: {SERIALBUFFER}{L}
SERIALPORT ADD COM5 SERIALPORT SEND COM5 Recieved!
SERIALPORT ADD COM5 COLOR DEFAULT

The above code creates a new serial port channel on COM5 with a baud rate of 9600. Setting the serial buffer to default routes all received data to the SERIALBUFFER previously mentioned. The following 4 lines of code add events to the data received event and will execute them all chronologically; set the text color to red, print the received data to the console, send the message “Recieved!” and new line, and finally reset the console color to its default setting (black background, white foreground).

It’s important to understand the serial port channel automatically routes the received data to the designated buffer, then proceeds to execute each command chronologically. By default, there are no commands to execute. If throughout your code you have added commands to execute but want to disable the automatic execution of commands when data is received, you can toggle the feature:

SERIALPORT TOGGLE PORTNAME

This will set the serial port channel’s data received event to either automatically run all commands, or to not run all commands. This *will not disable* the automatic routing of data to the serial buffer – only the automatic execution of commands in the command list when data is received.

Basic Network Commands

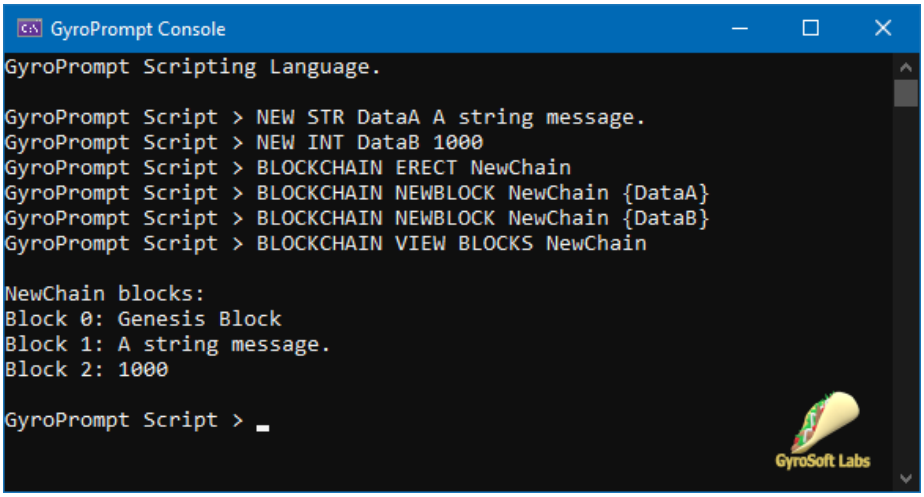
When you ping, you can have it print straight to the console or have the output placed into a string variable. Below is the ping syntax:

PING ADDRESS TIMEOUT COUNT DATA
PING _OUTPUT ADDRESS TIMEOUT COUNT DATA

The address is the IP address or internet URL you are pinging, timeout is how long in milliseconds, count is how many times to repeat the ping, and the data is the actual data being sent in the ping.

Working with Blockchain

The standard blockchain is simply formatted with a genesis block, where other blocks can be individually added with typed data and variables. Each newly appended block's hash matched the previous blocks. Below is a table of commands for interacting with and creating a blockchain:

Command Syntax				Description
BLOCKCHAIN	ERECT	NAME		Generates new blank blockchain with a genesis block.
BLOCKCHAIN	NEWBLOCK	NAME	DATA	Generates a new block, appending it to the specified blockchain.
BLOCKCHAIN	VIEW	MODE	NAME	View the specified blockchain in a human readable or serialized print.
BLOCKCHAIN	EXPORT	NAME		Serialize and output the blockchain into a file.

Blocks are capable of storing variables and lists. The export command can be especially useful if you intend to transmit or share the blockchain to another party and would like to keep a current copy. This way you can compare the hash values of the blockchain at the time it was transmitted to the returned version and manually ensure the blockchain's integrity. There are two view modes, **RAW** and **BLOCKS**. RAW displays a serialized version of the chain and this is less human readable, but show's each blocks hash value. The RAW mode is what would be exported. The BLOCKS mode shows the data in each block in a more human friendly manner. Below is an example of a very basic blockchain being created and displayed in BLOCKS mode: